Toward Fortran Level AMICA in Julia

Final Presentation

Valentin Morlock, 24.3.2025

Agenda

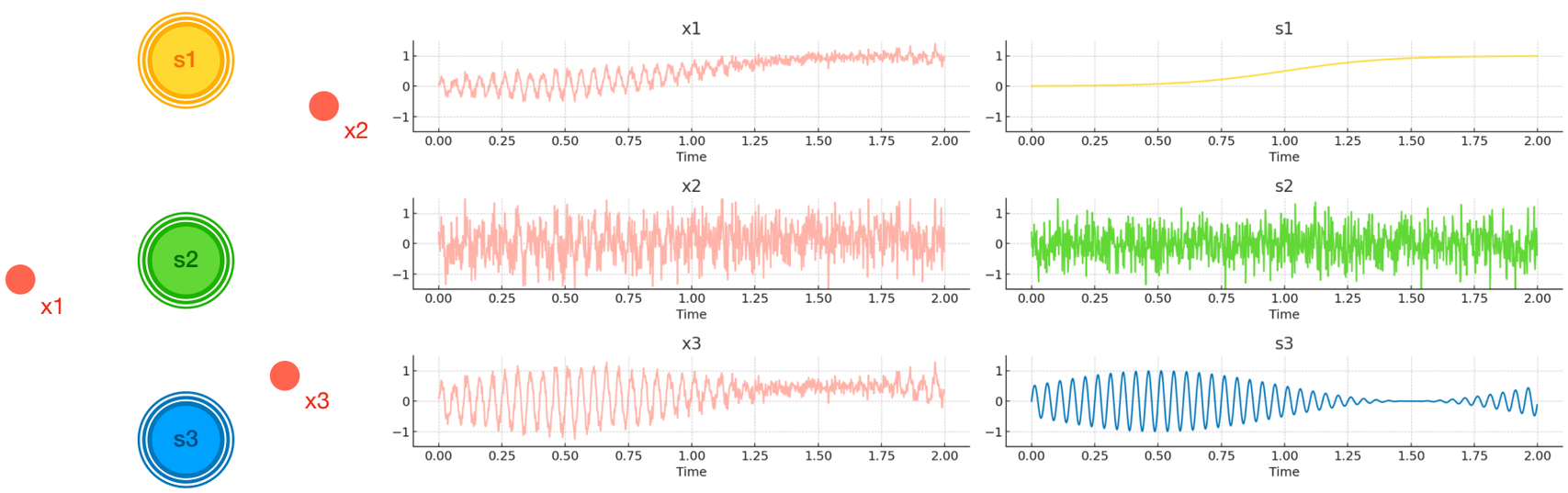
- Recap: ICA, Amica, Research Question
- Before this Thesis
- Recap: Approach
- Results
  - Correctness
  - Runtime
  - Memory
- Interpretation
- Limitations & Outlook

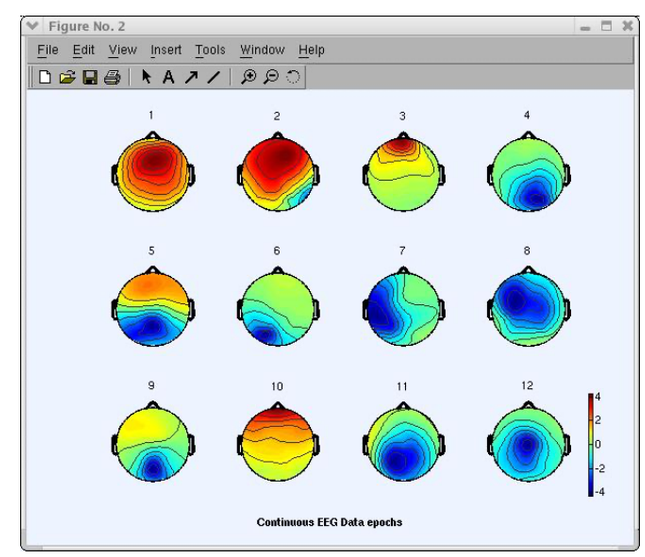
Independent Component Analysis

- ICA separates mixed signals into original sources by linear transformation from source signals s to mixtures x
  $x = A * s$ → estimate $W = A^{-1}$ to reverse mixing $s = x * W$

- Approached e.g. by maximizing entropy or maximizing likelihood

- Used to remove Artifacts (e.g. eye blinks, heartbeats) or separate brain sources from EEG signals

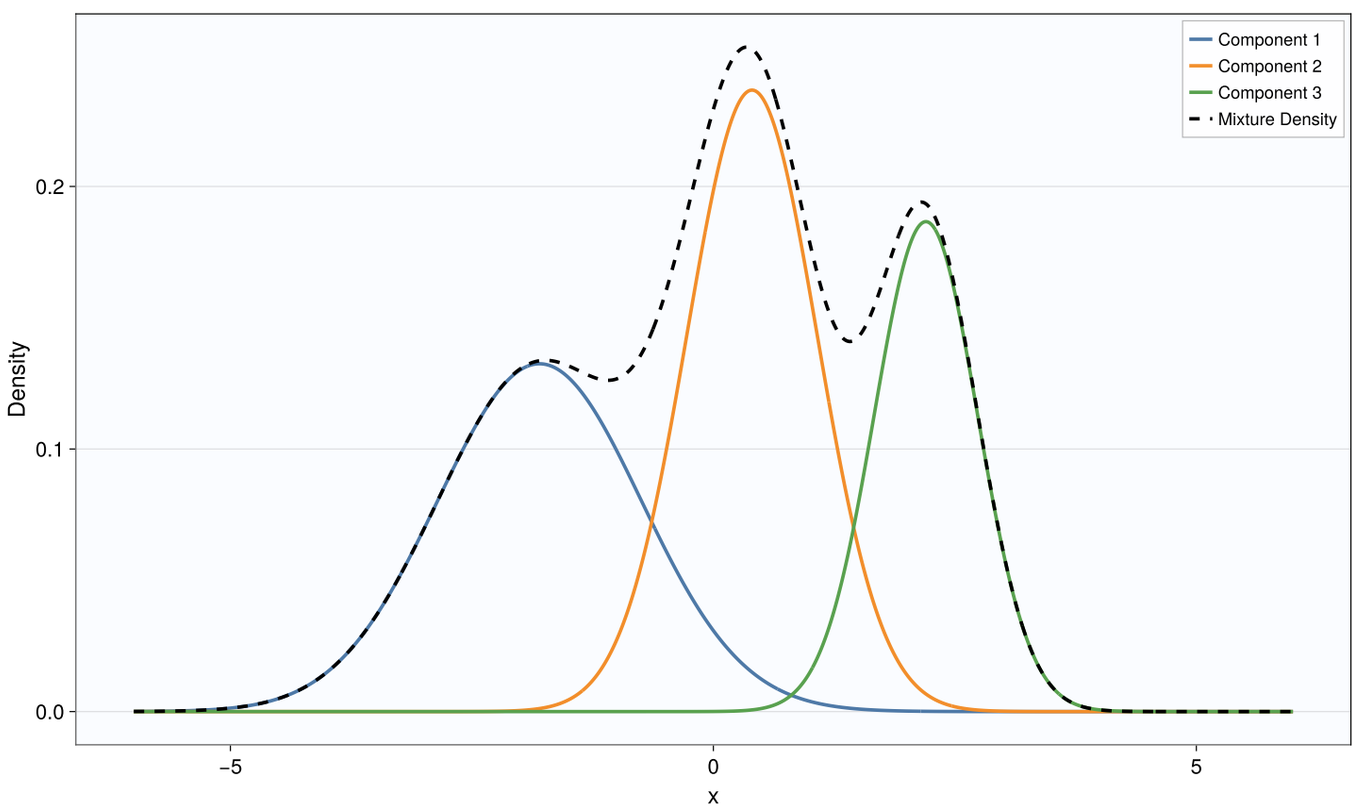


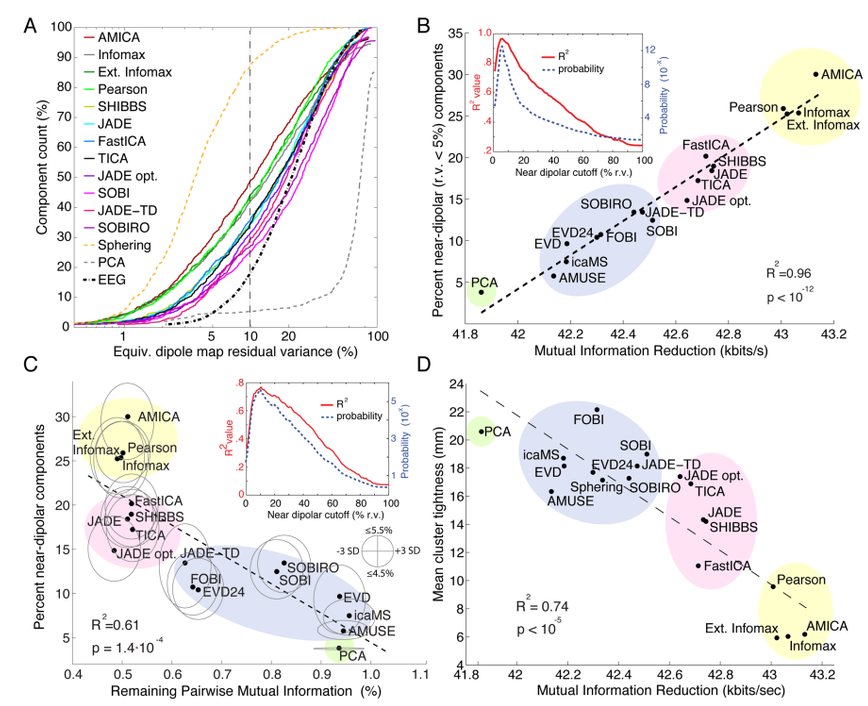


https://eeglab.org/tutorials/06_RejectArtifacts/RunICA.html

AMICA

- Maximum-likelihood based ICA
- Source densities modeled as Mixtures of Generalized Gaussians
- Multiple models for non-stationarity
- Found to be the best performing ICA algorithm for separating brain signals from non brain signals





Delorme, Arnaud, et al. "Independent EEG sources are dipolar." *PloS one* 7.2 (2012): e30135.

Problem & Research Question

- Three implementations with distinct problems

	Maintainability	Convergence	Performance
Fortran	--	++	++
Matlab	+	-	--
Julia	++	-	--

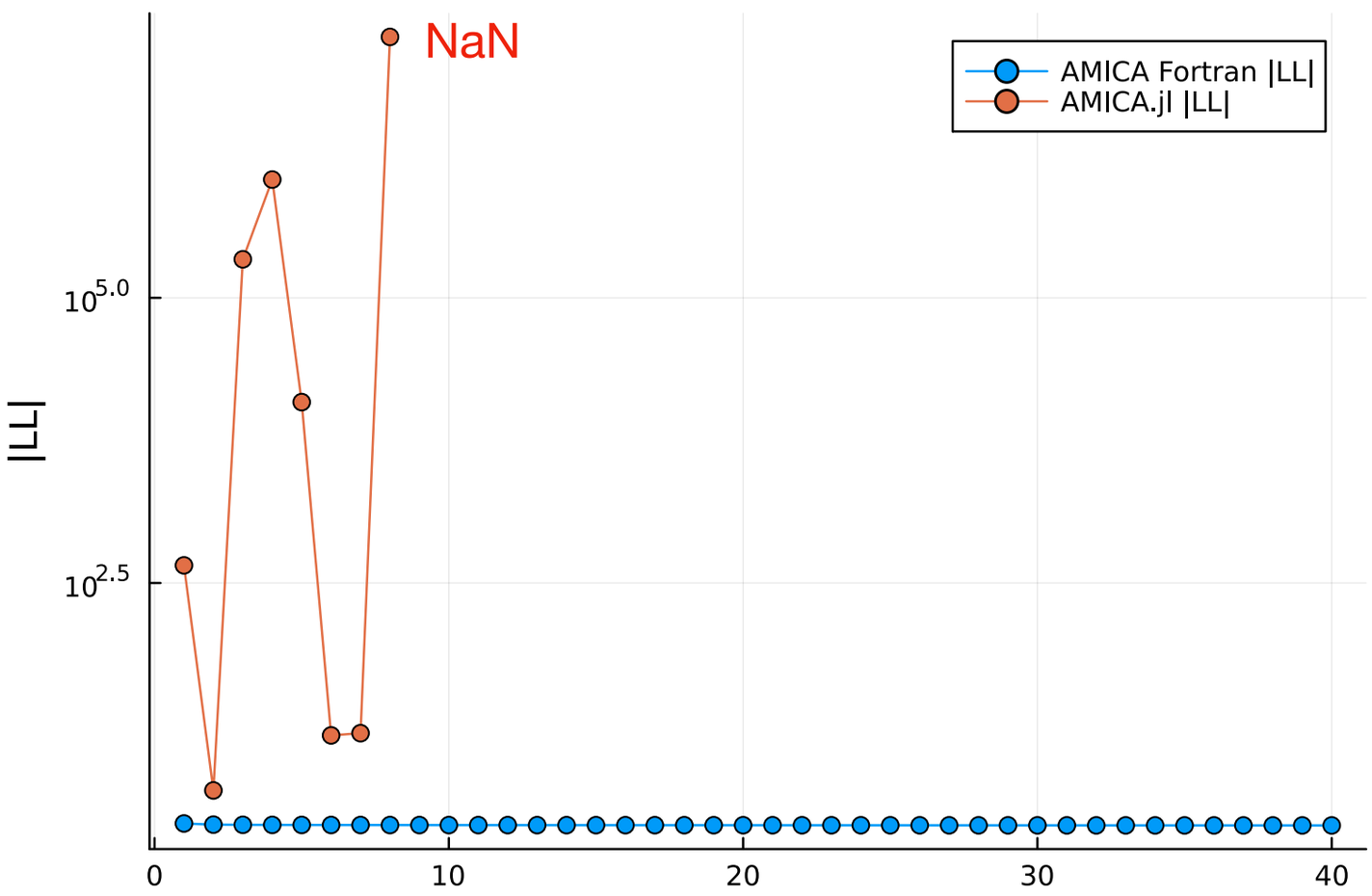
- Can an optimized Julia AMICA match Fortran in convergence and runtime efficiency?
- Which improvements (e.g. blockwise learning, GPU acceleration) show the strongest contribution?

Before this Thesis

- Iteration time roughly matches single-threaded Fortran algorithm, no parallelization, high memory consumption
- Results were tested against Matlab but not Fortran
- Issues with numerical stability

Before this Thesis

- Iteration time roughly matches single-threaded Fortran algorithm, no parallelization, high memory consumption
- Results were tested against Matlab but not Fortran
- Issues with numerical stability



Max. Memory GB	s / Iter
CPU ~ 7.5	~ 6,0

Dataset for all upcoming benchmarks:

Memorize.fdt
86,53MB Float32, therefore ~172MB Float64

(71 x 319500) 71 channel Sternberg dataset from Onton and Makeig 2006
https://sccn.ucsd.edu/~jason/amica15/Memorize.fdt

Testing

- Timing and allocation tracking with TimerOutputs.jl
- Automatic tests to compare outputs and intermediaries to Fortran
- GitHub Actions pipeline

→ Test failures can be narrowed down to individual calculations

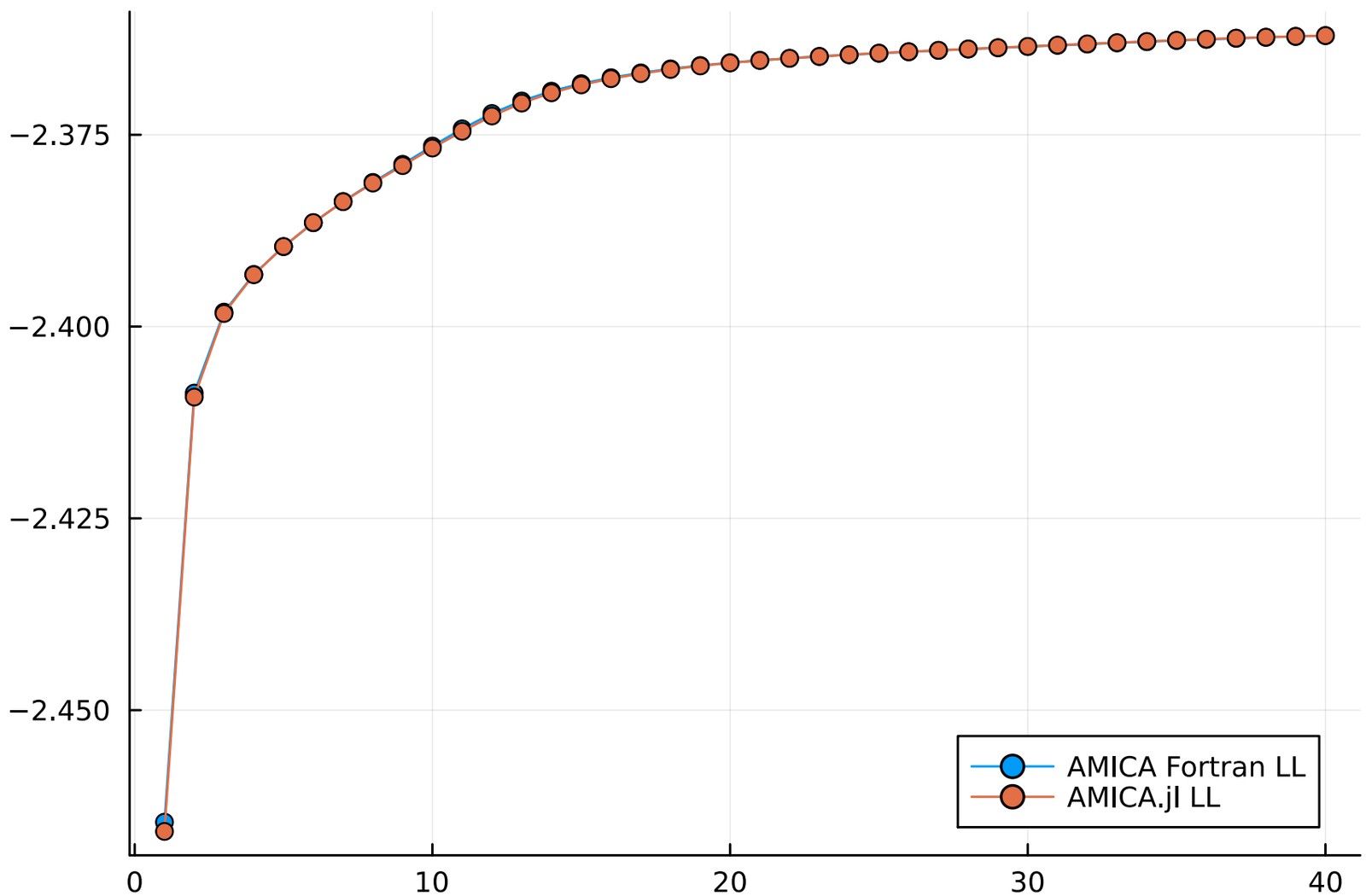
```
compare without newton: Test Failed at /home/fapra_morlock/hallo/Amica.jl/test/int
  Expression: myAmica.proportions ≈ alpha
   Evaluated: [0.3720550818120806 0.4629496027629765 0.36499531542494296; 0.344829
064445] ≈ [0.31004590151006944 0.38579133563581597 0.30416276285411736; 0.28735827
6720367]
```

Initial Cleanup

- ~15 algorithmic changes to match LL to Fortran
- Improved numerical stability by clamping small values to ε
- Used *claude code* to extract implementation details from Fortran

Initial Cleanup

- ~15 algorithmic changes to match LL to Fortran
- Improved numerical stability by clamping small values to ε
- Used *claude code* to extract implementation details from Fortran



Max. Memory GB		s / Iter
CPU	~ 7,5	~ 6,0
CPU	~ 3,5	~ 7,1

GPU Support

Broadcasts

- Efficient on CPU and GPU
- Unwanted allocations (e.g. sum, re-use calculations), partially solved using scratch arrays

```jl
fp = y_rho .* sign.(y) .* push_dimension(shape)
dlambda_numer = sum(z .* (fp .* y .- T(1.0)) .^ 2, dims=1)[1, :, :]
g .= sum(push_dimension(scale) .* z .* fp, dims=3)[:, :, 1]
```

GPU Support

Broadcasts

- Efficient on CPU and GPU
- Unwanted allocations (e.g. sum, re-use calculations), partially solved using scratch arrays

```jl
fp = y_rho .* sign.(y) .* push_dimension(shape)
dlambda_numer = sum(z .* (fp .* y .- T(1.0)) .^ 2, dims=1)[1, :, :]
g .= sum(push_dimension(scale) .* z .* fp, dims=3)[:, :, 1]
```

GPU Support

Kernels

- Re-use intermediaries without allocation
- Summing is hard (lock congestion, numerical stability)
- CPU performance

```
fp = y_rho[k, i, j] * sign(y[k, i, j]) * shape[i, j]
Atomix.@atomic dlambda_numer[i, j] += z[k, i, j] * (fp * y[k, i, j] - T(1.0))^2
Atomix.@atomic g[k, i, 1] += scale[i, j] * z[k, i, j] * fp
```

GPU Support

Kernels

- Re-use intermediaries without allocation
- Summing is hard (lock congestion, numerical stability)
- CPU performance

```jl
@kernel function g_dlambda_kernel!(
    g, dlambda_numer,
    @Const(y_rho), @Const(y), @Const(shape), @Const(scale),
    @Const(z), @Const(T)
)
    k, i, j = @index(Global, NTuple)
    fp = y_rho[k, i, j] * sign(y[k, i, j]) * shape[i, j]
    Atomix.@atomic dlambda_numer[i, j] += z[k, i, j] * (fp *
    y[k, i, j] - T(1.0))^2
    Atomix.@atomic g[k, i, 1] += scale[i, j] * z[k, i, j] * fp
end
g .= zero(T)
dlambda_numer .= zero(T)
kernel! = g_dlambda_kernel!(get_backend(y))
kernel!(g, dlambda_numer, y_rho, y, shape, scale, z, T,
ndrange=size(y))
```

GPU Support

Kernels

- Re-use intermediaries without allocation
- Summing is hard (lock congestion, numerical stability)
- CPU performance

```jl
@kernel function g_dlambda_kernel!(
    g, dlambda_numer,
    @Const(y_rho), @Const(y), @Const(shape), @Const(scale),
    @Const(z), @Const(T)
)
    k, i, j = @index(Global, NTuple)
    fp = y_rho[k, i, j] * sign(y[k, i, j]) * shape[i, j]
    Atomix.@atomic dlambda_numer[i, j] += z[k, i, j] * (fp *
    y[k, i, j] - T(1.0))^2
    Atomix.@atomic g[k, i, 1] += scale[i, j] * z[k, i, j] * fp
end
g .= zero(T)
dlambda_numer .= zero(T)
kernel! = g_dlambda_kernel!(get_backend(y))
kernel!(g, dlambda_numer, y_rho, y, shape, scale, z, T,
ndrange=size(y))
```

GPU Support

Initial Solution

- SingleModelAmica is generic over the kind of array
- Mostly broadcasts, two custom kernels
- Preparation functions run on CPU

```julia
# before
mutable struct SingleModelAmica{T} <:AbstractAmica
  location::Array{T, 2}
  shape::Array{T, 2}
  Lt::Array{T, 1}
  # ... further fields ...
end

# after
mutable struct SingleModelAmica{
  T,
  Array1<:DenseArray{T,1},
  Array2<:DenseArray{T,2},
  Array3<:DenseArray{T,3}
} <: AbstractAmica
  location::Array2
  shape::Array2
  Lt::Array1
  # ... further fields ...
end
```

	Max. Memory GB	s / Iter
CPU	~ 7,5	~ 6,0
CPU	~ 3,5	~ 7,1
GPU	~ 15,6	~ 0,1
CPU	~ 3,6	~ 6,6

Blockwise Processing

- Idea: most outputs are sums along samples, compute the intermediary in blocks and add to the output value
- Blocks can be parallelized across threads

```jl
# without blocks
scratch .= (z .* (y_rho .* sign.(y) .* shape))
dmu_numer .= sum(scratch, dims=1)[1, :, :]
```

Blockwise Processing

- Idea: most outputs are sums along samples, compute the intermediary in blocks and add to the output value
- Blocks can be parallelized across threads

```jl
# without blocks
scratch .= (z .* (y_rho .* sign.(y) .* shape))
dmu_numer .= sum(scratch, dims=1)[1, :, :]

# with blocks
dmu_numer .= zero(T)

for blocks in 1:num_blocks
  range = ((block-1)*block_size+1):(block*block_size)
  scratch .= (z[range] .* (y_rho[range] .* sign.(y[range]) .* shape[range]))
  dmu_numer .+= sum(scratch, dims=1)[1, :, :]
end
```

Blockwise Processing

- Idea: most outputs are sums along samples, compute the intermediary in blocks and add to the output value
- Blocks can be parallelized across threads

```jl
# without blocks
scratch .= (z .* (y_rho .* sign.(y) .* shape))
dmu_numer .= sum(scratch, dims=1)[1, :, :]

# with blocks
dmu_numer .= zero(T)

for blocks in 1:num_blocks
  range = ((block-1)*block_size+1):(block*block_size)
  scratch .= (z[range] .* (y_rho[range] .* sign.(y[ran
  shape[range]))
  dmu_numer .+= sum(scratch, dims=1)[1, :, :]
end
```

Max. Memory GB		s / Iter
CPU	~ 7,5	~ 6,0
CPU	~ 3,5	~ 7,1
GPU	~ 15,6	~ 0,1
CPU	~ 3,6	~ 6,6
GPU	~ 0,9	~ 0,5
CPU	~ 1,1	~ 4,9
CPU*64	~ 2,8	~ 0.6

Blocksize: 1000

Benchmarks

Setup

- Three EEG datasets
  - 19 channel, 172,704 sample: “Cognitive Workload" (Barras & Booth 2026)
  - 71 channel, 319,500 sample: „Memorize" (Onton & Makeig 2006)
  - 128 channel, 1,260,379 sample: "EEG Eye Tracking" (Schepers et al. 2026)

Benchmarks

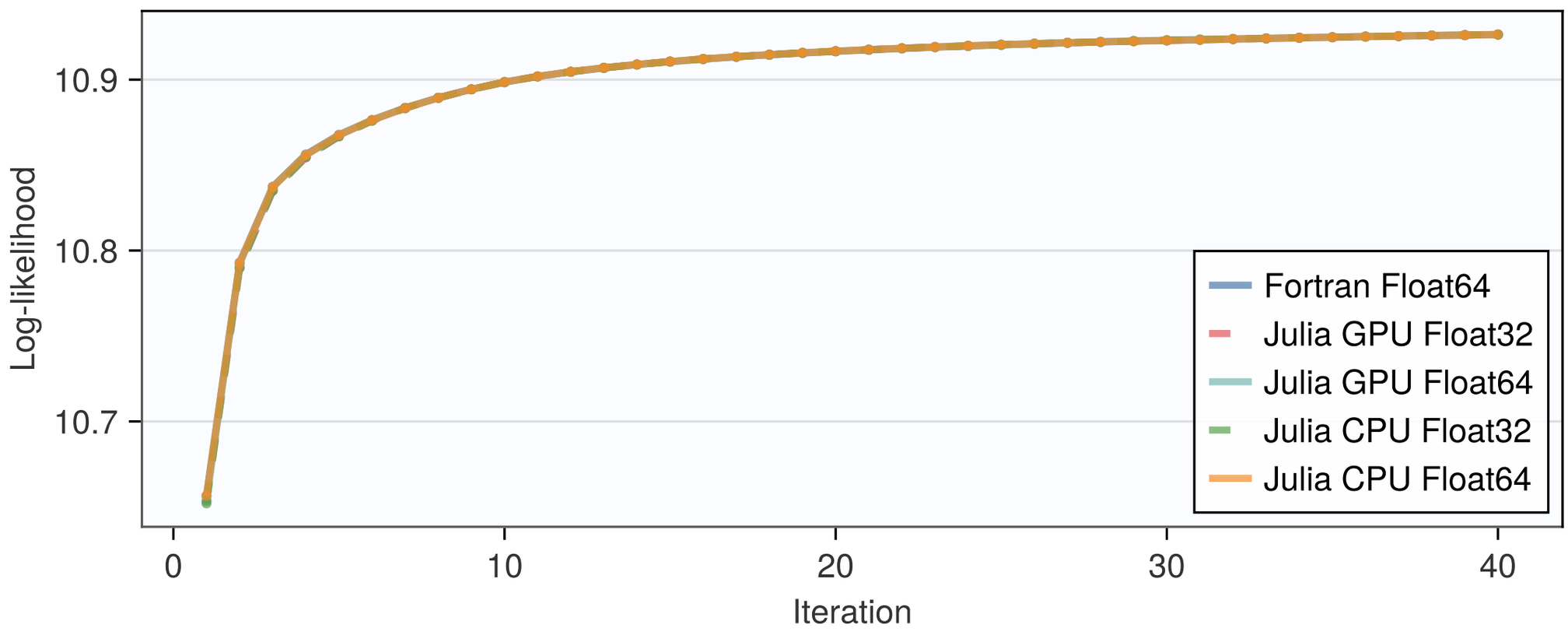
Setup

- Three EEG datasets
  - 19 channel, 172,704 sample: “Cognitive Workload" (Barras & Booth 2026)
  - 71 channel, 319,500 sample: „Memorize" (Onton & Makeig 2006)
  - 128 channel, 1,260,379 sample: "EEG Eye Tracking" (Schepers et al. 2026)
- Running on the group’s server
  - 2x AMD EPYC 7452 32 Core Processor, 354GB Memory
  - 1x NVIDIA RTX A6000, 49GB Memory
- 259 distinct parameterizations, 6 runs per parameterization

Results

Correctness

- Matching LL for three datasets and 40 iterations
- Independent of implementation and precision

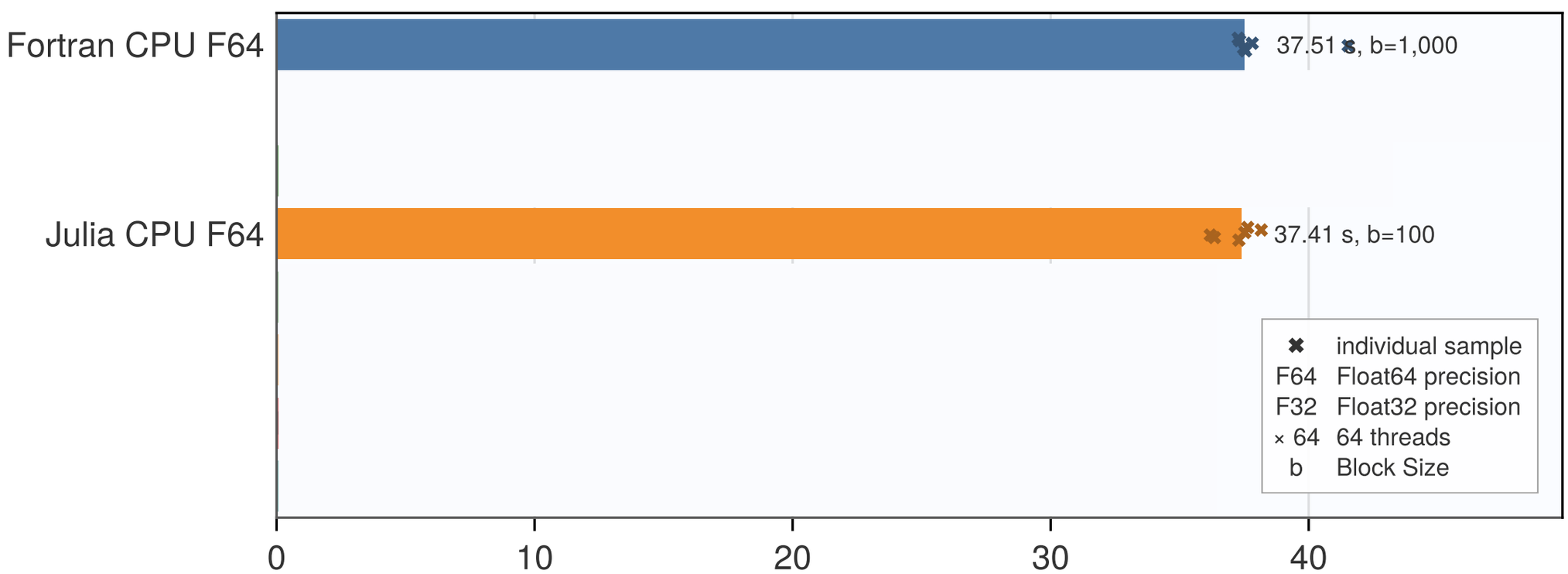


„EEG Eye Tracking“ Dataset

Results

Runtime

- Nearly identical single thread runtime

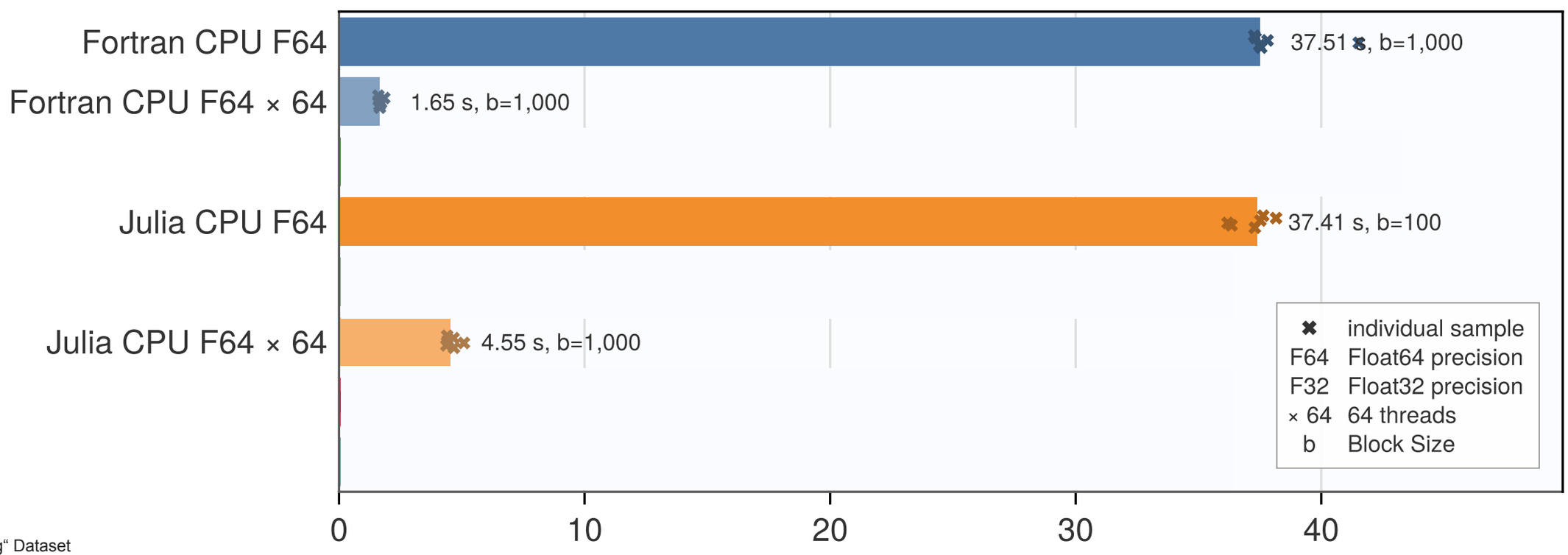


„EEG Eye Tracking“ Dataset

Results

Runtime

- Nearly identical single thread runtime
- Fortran is more efficient in multithreading

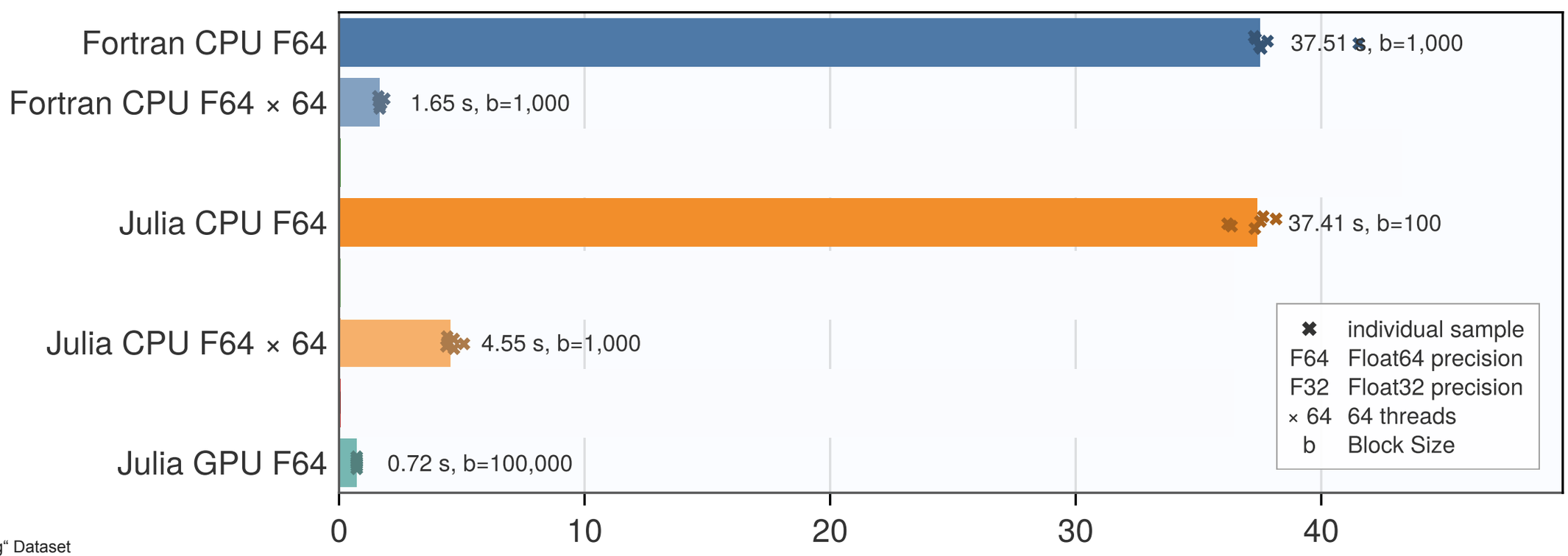


„EEG Eye Tracking“ Dataset

Results

Runtime

- Nearly identical single thread runtime
- Fortran is more efficient in multithreading
- Julia on the GPU is around ~2.2x faster than 64-thread Fortran

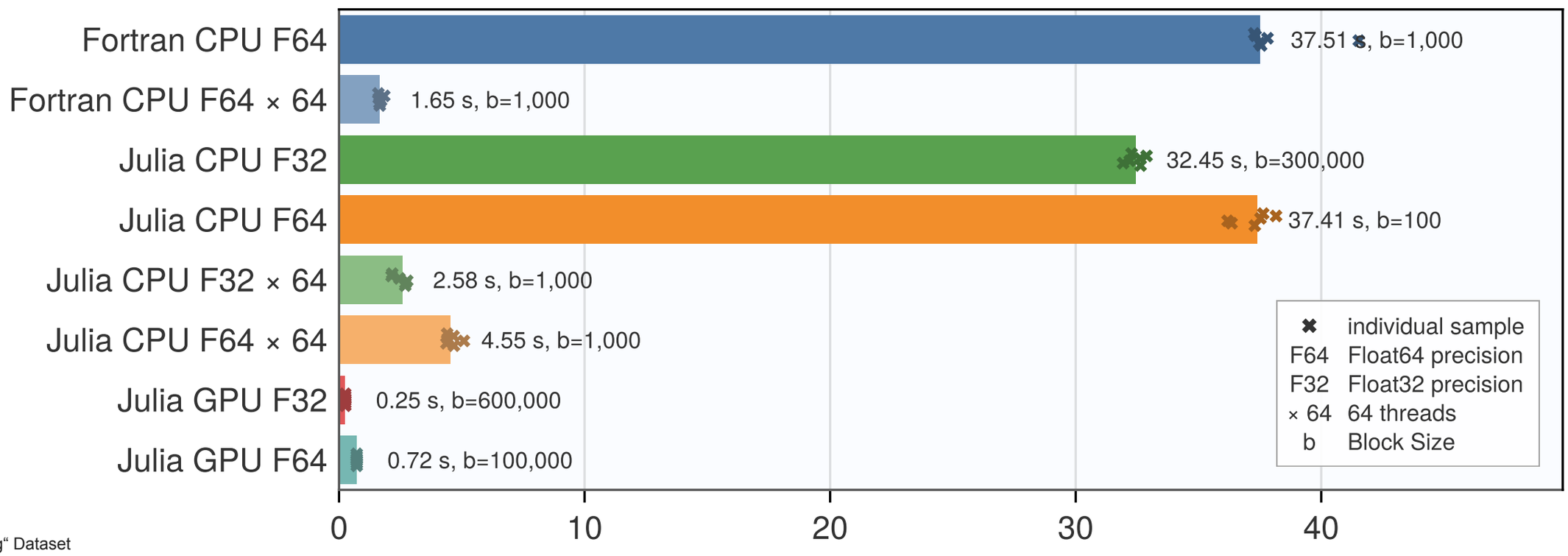


„EEG Eye Tracking“ Dataset

Results

Runtime

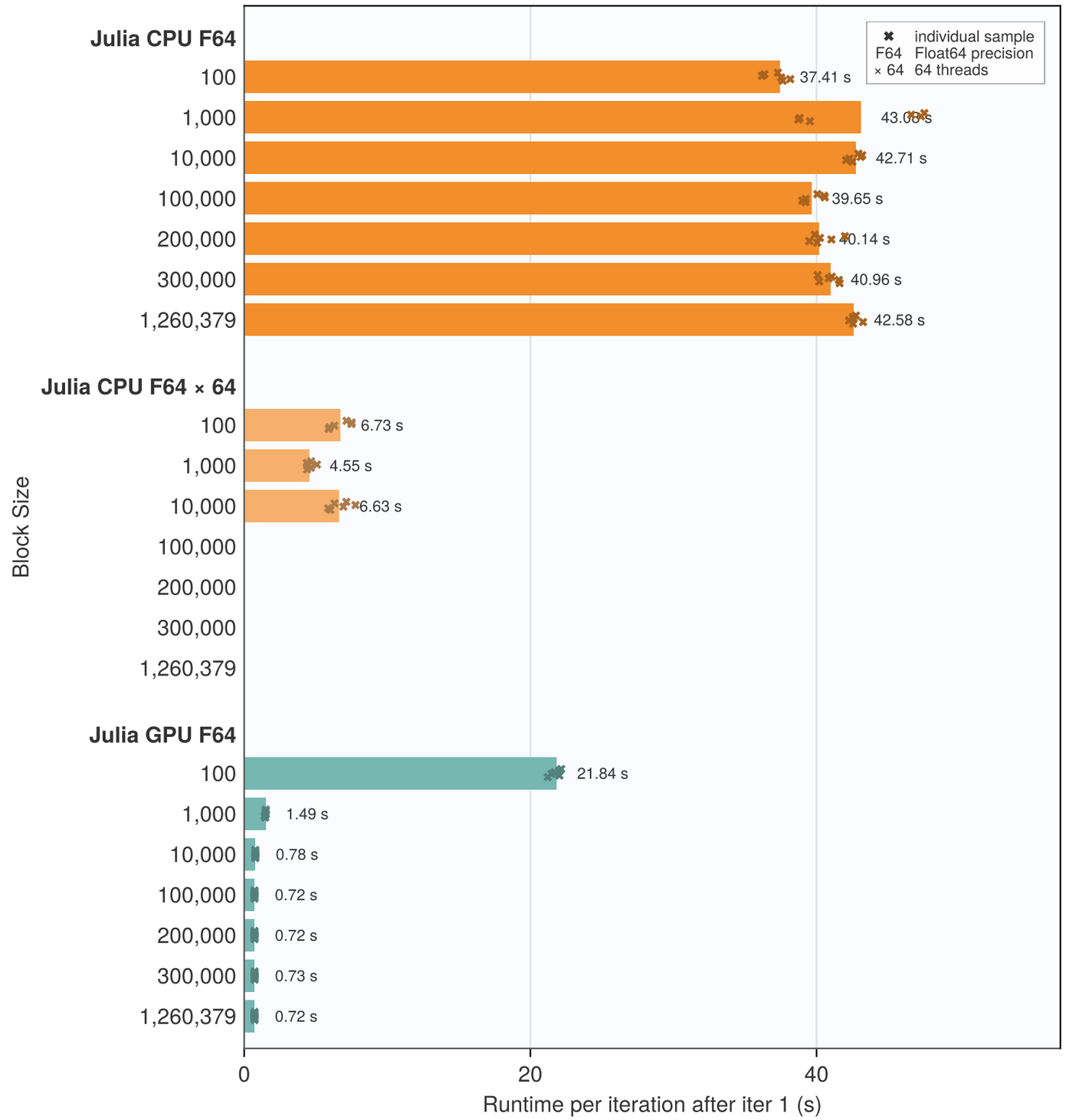
- Nearly identical single thread runtime
- Fortran is more efficient in multithreading
- Julia on the GPU is around ~2.2x faster than 64-thread Fortran
- Further speedup through 32-bit precision, ~6.6x faster on GPU than fastest Fortran



Results

Runtime - Block Size

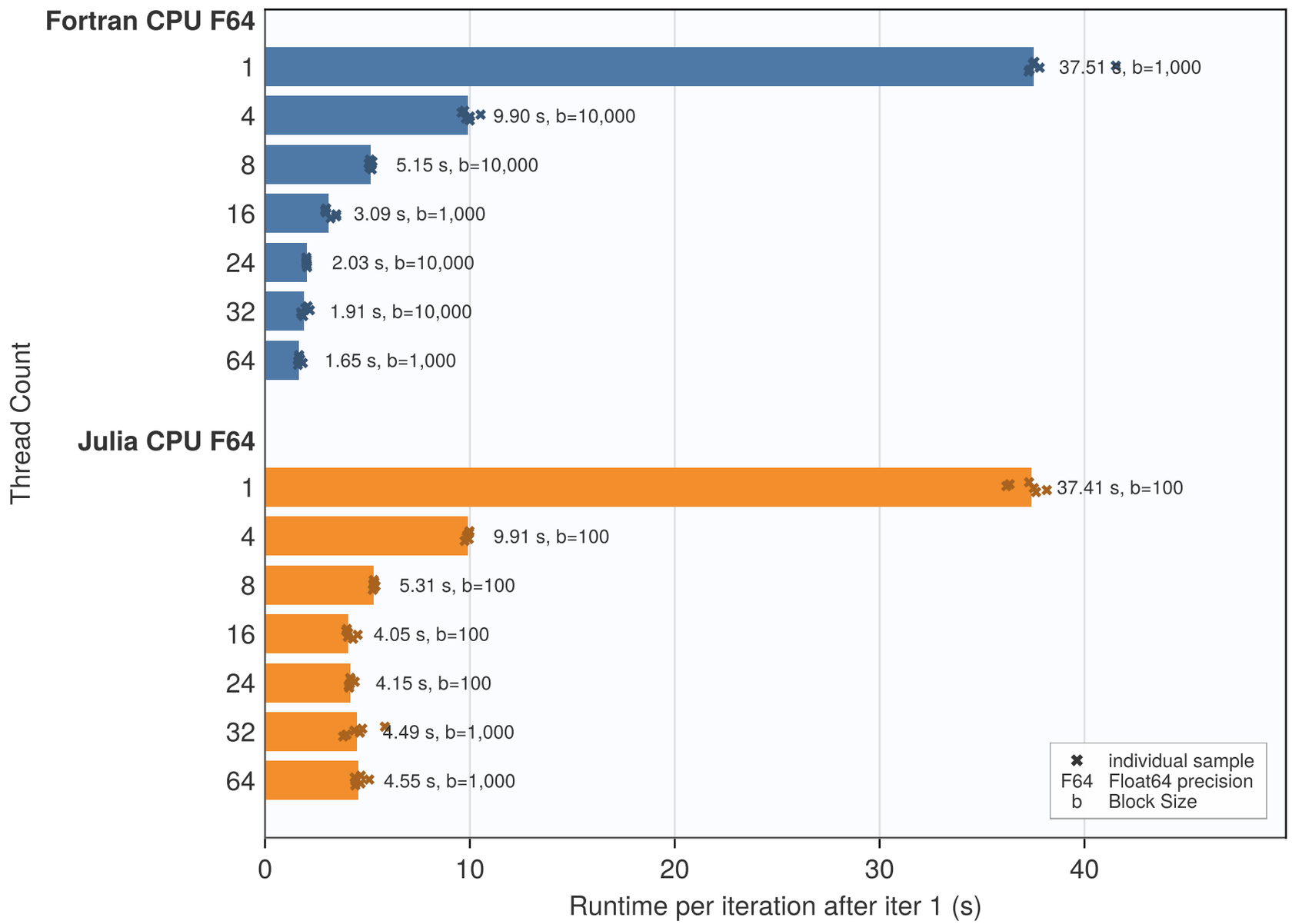
- Block size plays minor role for runtime
- Very small block sizes perform worse on GPU
- Large block sizes perform worse on CPU



Results

Runtime - Thread Count

- Results diminish around 16 threads

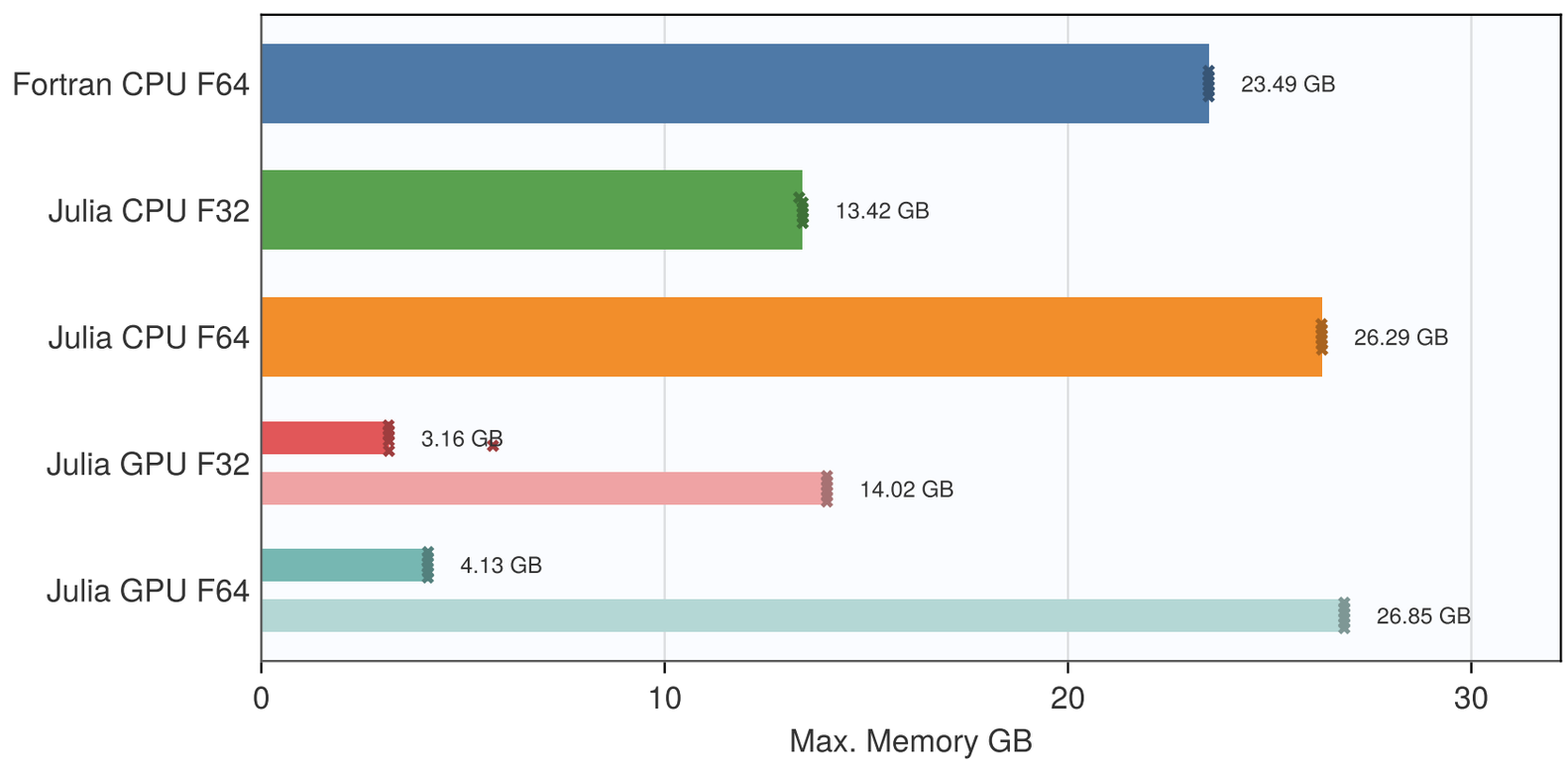


„EEG Eye Tracking“ Dataset

Results

Memory

- Fortran remains the most memory efficient implementation
- Offset increases for larger datasets
- GPU variant uses almost static amount of CPU memory
- Blockwise processing & 32-bit precision effectively reduce memory

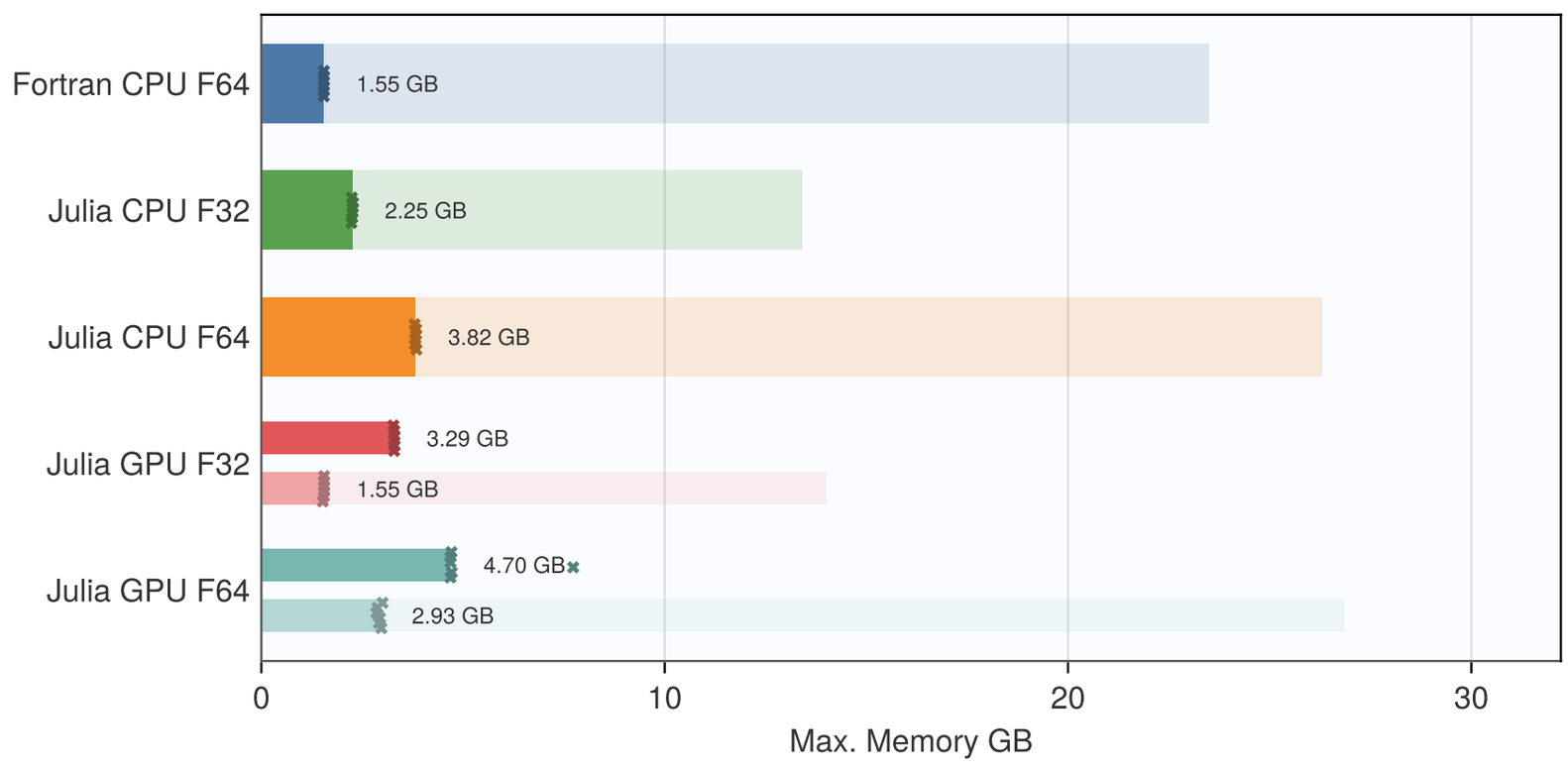


„EEG Eye Tracking“ Dataset

Results

Memory

- Fortran remains the most memory efficient implementation
- Offset increases for larger datasets
- GPU variant uses almost static amount of CPU memory
- Blockwise processing & 32-bit precision effectively reduce memory



„EEG Eye Tracking“ Dataset

Interpretation

- AMICA.jl is now a **credible** and **accessible** alternative to the Fortran version, **competitive in runtime and memory** use
- GPU acceleration is feasible and strongly accelerates AMICA, reducing time and potentially cost to run AMICA
- Blockwise processing effectively reduces memory use, enables running more subjects in parallel
- Test suite will catch future regressions
- Code is more compact and accessible than Fortran

Limitations & Outlook

- AMICA.jl misses advanced Fortran features, e.g. multi model
- CPU-only Julia still behind Fortran in terms of runtime
- Test suite does not cover every code path
- GPU support was mainly tested on CUDA
- Benchmarks cover only a limited set of datasets and setups